MEPCO SCHLENK ENGINEERING COLLEGE (AUTONOMOUS), SIVAKASI

Department of Computer Science and Engineering

19CSP01 – Free Open Source Software

Assignment 2: Package Development & Version Control using Git and GitHub

Question

Make a sample package of your own. Host it in GIT. Check out any file and perform changes and push it again in the GIT repository.

Aim

To design and create a sample modular Python arithmetic package ('calculator'), initialize it under Git version control, host it publicly on GitHub, check out the repository to modify an existing source file, and push the updated commit back to GitHub.

Introduction

Free and Open Source Software (FOSS) thrives on decentralized collaboration, version control, and modular code reusability. Git is a distributed version control system designed to handle everything from small to large projects with speed and efficiency. GitHub is a cloud platform that hosts Git repositories and provides collaborative tools like code review, branch protection, and commit tracking.

A Python package is an organized directory containing an __init__.py file and multiple module files along with a setup.py distribution script. In this experiment, a modular math package named 'calculator' is created, tracked locally using Git, published to GitHub, and updated through standard clone-modify-push workflow.

System Requirements

- Operating System: Windows 11 (64-bit)
- Version Control Tool: Git version 2.52.0.windows.1
- Programming Language: Python 3.x with setuptools
- Code Editor: Visual Studio Code / Terminal
- Remote Repository Platform: GitHub (https://github.com/vishwaa-thangaraja)

Package Structure & Design

The created Python package 'calculator' consists of the following modular files:

- calculator/__init__.py: Module initializer exposing arithmetic functions.
- calculator/operations.py: Core module implementing add, subtract, multiply, and divide functions.
- setup.py: Packaging and distribution metadata file configured with setuptools.
- README.md: Complete package documentation, installation guidelines, and usage examples.
- .gitignore: Standard ignore configuration excluding Python cache and build directories.

Procedure and Execution Steps

Step 1: Configure Git (First-Time Setup)

Before initiating commits, the global user name and email were configured to establish developer identity across all Git logs.

$ git config --global user.name "Vishwaa T"

$ git config --global user.email "vishwaat27@gmail.com"

$ git config --list



Figure 1: Global Git User Configuration and List Output

Step 2: Create the Sample Python Package Directory

The project workspace and package directories were created with standard structure:

$ mkdir mypackage

$ cd mypackage

$ mkdir calculator

$ tree /F

```
D:\Documents\SEM V\Free open source software\Assignment\2\mypackage>tree /F
Folder PATH listing for volume New Volume
Volume serial number is 6664-AA30
D:.
    .gitignore
    README.md
    setup.py

└───calculator
        operations.py
        __init__.py
```

Figure 2: Directory Tree Structure of 'mypackage'

Step 3: Write the Package Source Code

The core mathematical logic was implemented in 'calculator/operations.py', exposed via 'calculator/__init__.py', and packaged via 'setup.py':

$ type calculator\operations.py

$ type setup.py

```
D:\Documents\SEM V\Free open source software\Assignment\2\mypackage>type calculator\operations.py
# calculator/operations.py
"""
Core arithmetic operations module.
"""
def add(a, b):
    return a + b
def subtract(a, b):
    return a - b
def multiply(a, b):
    return a * b
def divide(a, b):
    if b == 0:
        raise ValueError("Cannot divide by zero.")
    return a / b

D:\Documents\SEM V\Free open source software\Assignment\2\mypackage>type setup.py
from setuptools import setup, find_packages

setup(
    name="calculator",
    version="1.0.0",
    description="A simple arithmetic calculator package for Python",
    author="Vishwaa Thangaraja",
    author_email="vishwaat27@gmail.com",
    packages=find_packages(),
    classifiers=[
        "Programming Language :: Python :: 3",
        "License :: OSI Approved :: MIT License",
        "Operating System :: OS Independent",
    ],
    python_requires=">=3.6",
)
```

Figure 3: Source Code Implementation in Visual Studio Code

Step 4: Initialise a Local Git Repository

A new Git repository was initialized inside 'mypackage', and a .gitignore file was configured to prevent tracking compiled bytecode and temporary metadata:

$ git init

$ type .gitignore

```
D:\Documents\SEM V\Free open source software\Assignment\2\mypackage>git init
Initialized empty Git repository in D:/Documents/SEM V/Free open source software/Assignment/2/mypackage/.git/

D:\Documents\SEM V\Free open source software\Assignment\2\mypackage>type .gitignore
__pycache__/
*.py[cod]
*$py.class
*.egg-info/
dist/
build/
.pytest_cache/
```

Figure 4: Initializing Local Git Repository and .gitignore Configuration

Step 5: Stage and Commit the Files

All package files were added to the staging index, inspected via git status, and committed to the local repository as the initial baseline commit:

$ git add .

$ git status

$ git commit -m "Initial commit: Add calculator package"

```
D:\Documents\SEM V\Free open source software\Assignment\2\mypackage>git add .
warning: in the working copy of '.gitignore', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'README.md', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'calculator/__init__.py', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'calculator/operations.py', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'setup.py', LF will be replaced by CRLF the next time Git touches it

D:\Documents\SEM V\Free open source software\Assignment\2\mypackage>git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   .gitignore
        new file:   README.md
        new file:   calculator/__init__.py
        new file:   calculator/operations.py
        new file:   setup.py


D:\Documents\SEM V\Free open source software\Assignment\2\mypackage>git commit -m "Initial commit: Add calculator package"
[master (root-commit) c9f92b2] Initial commit: Add calculator package
 5 files changed, 69 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 README.md
 create mode 100644 calculator/__init__.py
 create mode 100644 calculator/operations.py
 create mode 100644 setup.py
```

Figure 5: Staging All Files and Executing Initial Commit

Step 6: Create GitHub Remote Repository and Push

A new public repository named 'mypackage' was created under the GitHub account 'vishwaa-thangaraja'. The local repository branch was set to main, linked to the remote URL, and pushed:

$ git branch -M main

$ git remote add origin https://github.com/vishwaa-thangaraja/mypackage.git

$ git push -u origin main

```
D:\Documents\SEM V\Free open source software\Assignment\2\mypackage>git branch -M main

D:\Documents\SEM V\Free open source software\Assignment\2\mypackage>git remote add origin https://github.com/vishwaa-thangaraja/mypackage.git

D:\Documents\SEM V\Free open source software\Assignment\2\mypackage>git push -u origin main
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 12 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (8/8), 1.32 KiB | 30.00 KiB/s, done.
Total 8 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/vishwaa-thangaraja/mypackage.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Figure 6: Pushing the Initial Package to Remote GitHub Repository

Step 7: Check Out Repository and Perform Changes

To simulate the distributed collaborative development cycle, the remote repository was cloned into 'mypackage_clone'. The module 'calculator/operations.py' was modified to add a new 'power(a, b)' arithmetic function, and the change was verified using git diff:

$ git clone https://github.com/vishwaa-thangaraja/mypackage.git mypackage_clone

$ cd mypackage_clone

$ git diff

```
D:\Documents\SEM V\Free open source software\Assignment\2\mypackage>cd ..

D:\Documents\SEM V\Free open source software\Assignment\2>git clone https://github.com/vishwaa-thangaraja/mypackage.git mypackage_clone
Cloning into 'mypackage_clone'...
remote: Enumerating objects: 8, done.
remote: Counting objects: 100% (8/8), done.
remote: Compressing objects: 100% (8/8), done.
remote: Total 8 (delta 0), reused 8 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (8/8), done.

D:\Documents\SEM V\Free open source software\Assignment\2>cd mypackage_clone

D:\Documents\SEM V\Free open source software\Assignment\2\mypackage_clone>git diff
diff --git a/calculator/__init__.py b/calculator/__init__.py
index 7ff9e51..26621eb 100644
--- a/calculator/__init__.py
+++ b/calculator/__init__.py
@@ -3,6 +3,6 @@
 Calculator Package
 Basic arithmetic operations module.
 """
-from .operations import add, subtract, multiply, divide
+from .operations import add, subtract, multiply, divide, power
 __version__ = "1.0.0"
-__all__ = ["add", "subtract", "multiply", "divide"]
+__all__ = ["add", "subtract", "multiply", "divide", "power"]
diff --git a/calculator/operations.py b/calculator/operations.py
index 76cc836..7bbef22 100644
--- a/calculator/operations.py
+++ b/calculator/operations.py
@@ -12,3 +12,5 @@ def divide(a, b):
         raise ValueError("Cannot divide by zero.")
     return a / b
+def power(a, b):
+    return a ** b

D:\Documents\SEM V\Free open source software\Assignment\2\mypackage_clone>
```

Figure 7: Cloning Repository and Inspecting Code Differences using git diff

Step 8: Stage, Commit, and Push the Updated File

The modified files ('operations.py' and '__init__.py') were staged, committed with a descriptive message, and pushed to the remote GitHub main branch:

$ git add calculator/operations.py calculator/__init__.py

$ git commit -m "feat: Add power() function to calculator package"

$ git push origin main

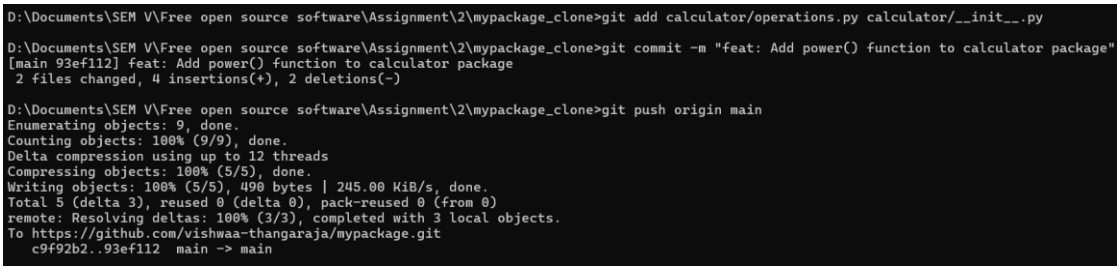


Figure 8: Committing and Pushing the New Feature to Remote GitHub

Step 9: Verify Commit History

The repository commit history was verified both locally via git log and remotely on GitHub, confirming the full chronology of changes:

$ git log --oneline

$ git log --stat

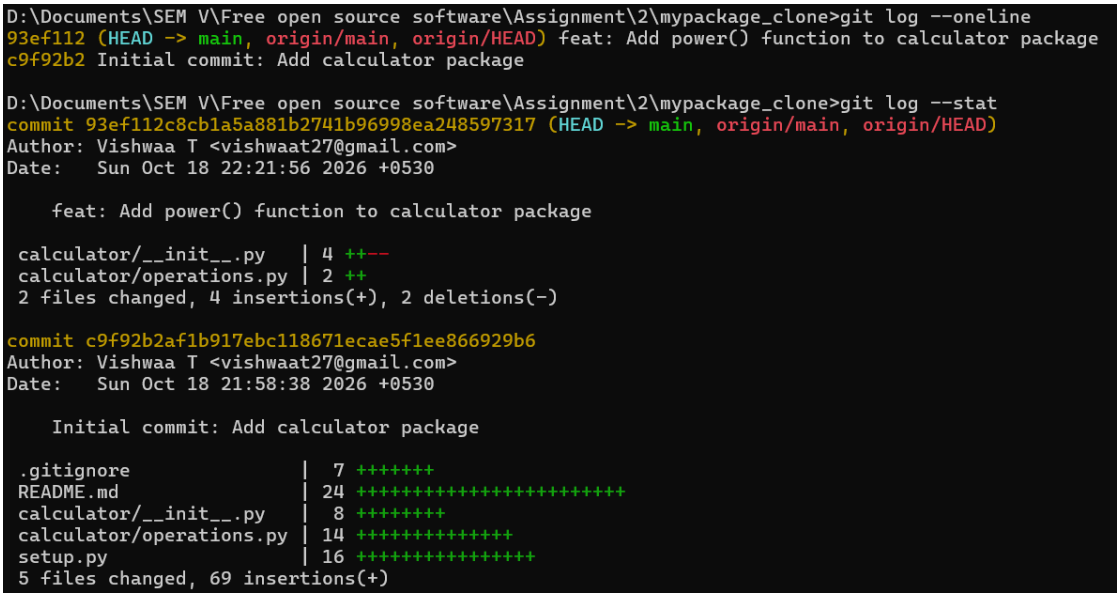


Figure 9: Commit Log Verification on GitHub / Terminal

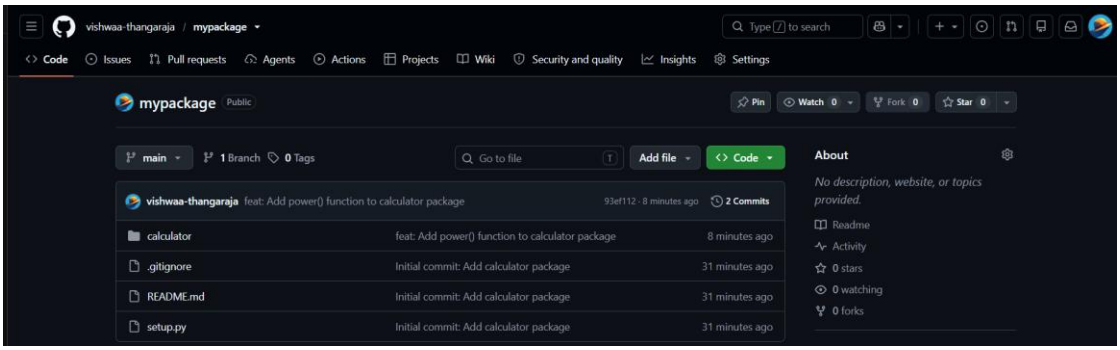


Figure 10: Complete Commit History Record on Remote Repository

Repository Details

- Repository Name: mypackage
- GitHub Remote URL: https://github.com/vishwaa-thangaraja/mypackage

Output / Result

A custom Python package named 'calculator' was successfully created, structured with standard packaging guidelines, and version-controlled using Git. The package was hosted on a public GitHub repository, checked out into a new workspace, modified by incorporating the power() exponentiation function, and successfully pushed back to the remote repository. The commit logs confirm seamless version history tracking.

Conclusion

This experiment successfully demonstrated the end-to-end lifecycle of FOSS development, package authoring, and distributed version control using Git and GitHub. Fundamental practices such as repository initialization, staging, atomic commits, remote synchronization, code diff inspection, and collaborative branching workflows were exercised and verified.